

♥ Like

📌 Save

🔗 Share



@thedevspaceio

CSS Functions Cheatsheet



calc()

Performs math with mixed units.

Syntax:

```
calc(<expression>)
```

css



```
1  div.container {  
2    width: calc(100% - 48px);  
3  }  
4  
5  .article-title {  
6    font-size: calc(1rem + 0.5vw);  
7  }
```

Always include spaces around operators (+, -, *, /) for broadest compatibility. Useful for layouts and spacing.

var()

Use CSS variables.

Syntax:

```
var(--name, <fallback>)
```

css



```
1  :root {  
2    --accent: #06b6d4;  
3    --space: 16px;  
4  }  
5  
6  .header {  
7    color: var(--accent, #06b6d4);  
8    padding: var(--space, 16px);  
9  }
```

Fallback is optional but recommended when the variable may be unset.

Custom properties cascade and can be changed at runtime.

clamp()

Clamp returns a relative value between a min and max constraints.

Syntax:

```
clamp(<min>, <preferred>, <max>)
```

css



```
1 h1.site-title {  
2   font-size: clamp(1rem, 2.5vw, 1.25rem);  
3 }  
4  
5 .grid {  
6   gap: clamp(8px, 2vw, 32px);  
7 }
```

Useful for responsive typography and responsive spacing without media queries.

min(), max()

Pick the minimum or maximum of several values.

Syntax:

`min(a, b, ...)` returns the smallest.

`max(a, b, ...)` returns the largest.

css



```
1  .card {  
2    width: min(60ch, 100%);  
3  }  
4  
5  .sidebar {  
6    height: max(40vh, 300px);  
7  }
```

Values can be mixed units; calculation happens at computed value time.

rgb() / rgba()

Used for sRGB color values and alpha transparency.

css



```
1 h2.section-title {  
2   color: rgb(255, 99, 71);  
3 }  
4  
5 .banner {  
6   background: rgba(96, 165, 250, 0.08);  
7 }
```

hsl() / hsla()

Hue, saturation, lightness notation. Supports alpha and slash-separated alpha syntax.

css



```
1 .banner {  
2   color: hsl(210 100% 50% / 0.9);  
3 }
```

url()

Reference external or local resources for images, fonts, etc.

css



```
1  .hero {  
2    background-image: url("/img/hero.png");  
3    background-size: cover;  
4  }
```

linear-gradient()

Gradients produce image values and can be used wherever images are allowed (backgrounds, masks, etc.).

`linear-gradient()` creates a gradient along a straight line.

css



```
1 .banner {  
2   background: linear-gradient(90deg, var(--color), var(--color-2))  
3 }
```



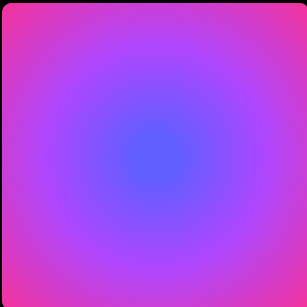
radial-gradient()

Expands outward from a center point; supports shape and size.

css



```
1 .glow {  
2   background: radial-gradient(  
3     circle at 50% 30%,  
4     rgba(96, 165, 250, 0.15),  
5     transparent 40%  
6   );  
7 }
```



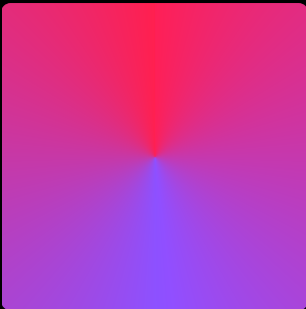
conic-gradient()

Angular gradient rotated around a center; useful for charts and loaders.

css



```
1 .pie {  
2   background: conic-gradient(  
3     from 0deg at 50% 50%,  
4     #60a5fa 0 25%,  
5     #7dd3fc 25% 50%,  
6     transparent 50% 100%  
7   );  
8 }
```



attr()

Use an element's attribute value in content or, in some experimental cases, property values.

Syntax:

```
attr(<name> [, <type-or-unit>])
```

html



```
1 <div class="badge" data-size="120">Badge</div>
```

css



```
1 .badge {  
2   width: attr(data-size px);  
3 }
```

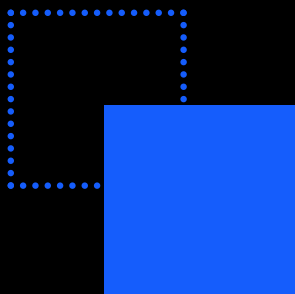
translate()

Move an element along the X and/or Y axis from its original position.

css



```
1 .translate {  
2   transform: translate(50%, 50%);  
3 }
```



rotate()

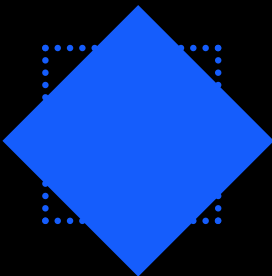
Rotate an element by an angle or turns.

css



```
1 .rotate {  
2   transform: rotate(45deg);  
3 }
```

Angles can be given in `deg`, `rad`, `turn`. Rotation occurs around the element's `transform-origin` (default `center`).



scale()

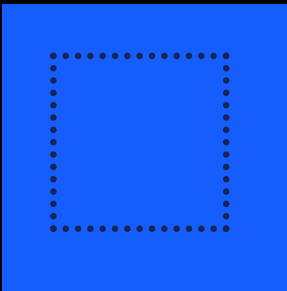
Scale an element along X and/or Y axes.

css



```
1  .scale-up {  
2    transform: scale(1.1); /* uniform scale */  
3  }  
4  
5  .scale-x {  
6    transform: scaleX(1.2);  
7  }  
8  
9  .scale-y {  
10   transform: scaleY(0.9);  
11 }
```

`scale()` multiplies element dimensions (1 is no change). Use `transform-origin` to control the scaling anchor point.



skew()

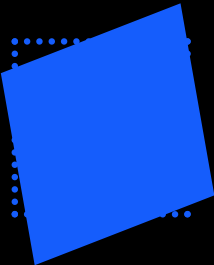
Skew an element along X and/or Y axes.

css



```
1 .skew-x {  
2   transform: skewX(10deg);  
3 }  
4  
5 .skew-y {  
6   transform: skewY(-20deg);  
7 }
```

`skew()` distorts the coordinate system, useful for subtle perspective effects.



Combining transforms

You can chain multiple transform functions.

css



```
1 .combo {  
2   transform: translateY(-4px) rotate(-2deg) scale(1.02);  
3 }
```

The functions are applied left-to-right. Changing order changes the result.

cubic-bezier()

Custom cubic Bézier timing function for transitions and animations.

Syntax:

```
cubic-bezier(x1, y1, x2, y2)
```

Example:

css



```
1 .button {  
2   transition: transform 240ms cubic-bezier(0.2, 0.9, 0.2, 1);  
3 }
```

steps()

Discrete step-based timing function.

Syntax:

```
steps(n, start|end)
```

Example:

css



```
1 .sprite {  
2   transition: background-position 600ms steps(6, end);  
3 }
```

Useful for frame-based sprite animations or snapping effects.

color-mix()

Blend two colors using color spaces.

Syntax:

```
color-mix(in srgb, colorA percentage, colorB)
```

Example:

css



```
1 .header-hero {  
2   background: linear-gradient(  
3     180deg,  
4     color-mix(in srgb, var(--accent) 70%, white),  
5     transparent 60%  
6   );  
7 }
```

`color-mix()` is new, check browser support before using it in production.